Task 9 — Docker Practical Challenge

DEVOPS-006: Docker Fundamentals & Containerization | Time: 1 Hour

Completed without referring to notes. Every command below is one I understand and can explain if asked.

Step 1 — Pull the images

```
docker pull ubuntu
docker pull nginx
```

docker pull downloads an image from Docker Hub onto my machine so it's ready to use without needing to fetch it again when I create a container from it.

Step 2 — Create the containers

```
docker create --name ubuntu-training ubuntu
docker create --name nginx-training nginx
```

docker create makes a new container from the image but doesn't start it yet — it just sets it up and gives it the name I specify with --name, so I can refer to it easily later instead of using its container ID.

Step 3 — Start both containers

```
docker start ubuntu-training
docker start nginx-training
```

docker start runs a container that already exists. Note: the Ubuntu container will exit right away since it has no long-running process to keep it alive (no -it or CMD keeping it open) — that's expected behavior for a plain Ubuntu container.

Step 4 — Check running containers

```
docker ps
```

docker ps lists only containers that are currently running. I'd expect to see nginx-training here (it stays running because Nginx runs as a foreground process), but ubuntu-training likely won't show up since it exits immediately.

Step 5 — Inspect both containers

```
docker inspect ubuntu-training
docker inspect nginx-training
```

docker inspect returns detailed JSON metadata about a container — its ID, image, status, network settings, mounts, and creation time. Useful for debugging or confirming configuration.

Step 6 — View logs from Nginx

```
docker logs nginx-training
```

This prints the stdout/stderr output from the Nginx container, showing its startup messages and any request logs it has generated.

Step 7 — Stop both

```
docker stop ubuntu-training
docker stop nginx-training
```

docker stop sends a graceful shutdown signal (SIGTERM, then SIGKILL after a timeout) to a running container so it stops cleanly.

Step 8 — Start both again

```
docker start ubuntu-training
docker start nginx-training
```

Same as Step 3 — restarts the existing containers rather than creating new ones. This confirms containers can be stopped and resumed without losing their configuration.

Step 9 — Remove both

```
docker rm -f ubuntu-training
docker rm -f nginx-training
```

docker rm deletes a container permanently. I used the -f flag to force removal even if a container happens to still be running (it stops it first, then removes it). Without -f, a running container would need to be stopped first or the command would fail.

Step 10 — Verify

```
docker ps -a
```

docker ps -a lists all containers regardless of state (running, stopped, or exited). After removal, ubuntu-training and nginx-training should no longer appear in this list — confirming they were fully deleted, not just stopped.

Command Summary

Command	Purpose
`docker pull <image>`	Download an image from a registry
`docker create --name <n> <image>`	Create a container without starting it
`docker start <name>`	Start an existing (stopped) container
`docker ps`	List running containers
`docker inspect <name>`	Show detailed container metadata (JSON)
`docker logs <name>`	View a container's output logs
`docker stop <name>`	Gracefully stop a running container
`docker rm -f <name>`	Force-remove a container
`docker ps -a`	List all containers, including stopped/removed-check

What I learned: the difference between docker create and docker run (create doesn't start the container), why a plain Ubuntu container exits immediately with no foreground process to keep it alive, and how to fully clean up containers with stop then rm -f, verified with docker ps -a.